NAME:RAGUL.M

REG NO:192524066

SUBJECT:PYTHON PROGRAMMING

CODE:CSA0802

CO:1 AT:1

Task 1 – Problem Analysis and Data Types

Aim: To analyse the milk payment problem and identify the required data types.

Theory: Problem analysis helps identify the inputs, processing steps and outputs. Milk quantity and rate can be represented using numeric data types. Variables store values, and the calculated payment is displayed as output.

Algorithm:

1. Identify the required inputs.
2. Identify the processing operation.
3. Select suitable data types.
4. Identify the expected output.

Pseudocode:

```
BEGIN
   INPUT milk_quantity
   INPUT rate
   payment = milk_quantity * rate
   DISPLAY payment
END
```

Test case: Quantity = 10 litres, Rate = ₹40; Expected payment = ₹400.

Result: The problem and its required data types are identified.



Task 2 – Values, Types and Numerical Precision

Aim: To understand values, data types and numerical precision in payment calculations.

Theory: Values represent the data used by a program. Data types define the kind of values stored in variables. Floating-point numbers can represent decimal values, but rounding errors may occur. The Decimal type can be used when accurate decimal calculations are important in financial applications.

Algorithm:

1. Read the milk quantity and rate.
2. Represent the values using appropriate numeric types.
3. Calculate the payment.
4. Display the result with suitable precision.

Pseudocode:

BEGIN

INPUT quantity

INPUT rate

payment = quantity * rate

ROUND payment TO 2 DECIMAL PLACES

DISPLAY payment

END

Test case: Quantity = 2.5 litres, Rate = ₹40; Expected payment = ₹100.00.

Result: Suitable data types and numerical precision are considered.



Task 3 – Variables and Constants

Aim: To understand the use of variables and constants in the milk payment system.

Theory: A variable stores a value that can change during program execution. A constant represents a value intended to remain fixed, such as a standard bonus rate. Proper variable names improve readability and make the program easier to maintain.

Algorithm:

1. Define variables for quantity and rate.
2. Define constants for fixed values, if required.
3. Store the input values.
4. Calculate and display the payment.

Pseudocode:

BEGIN

INPUT quantity

INPUT rate

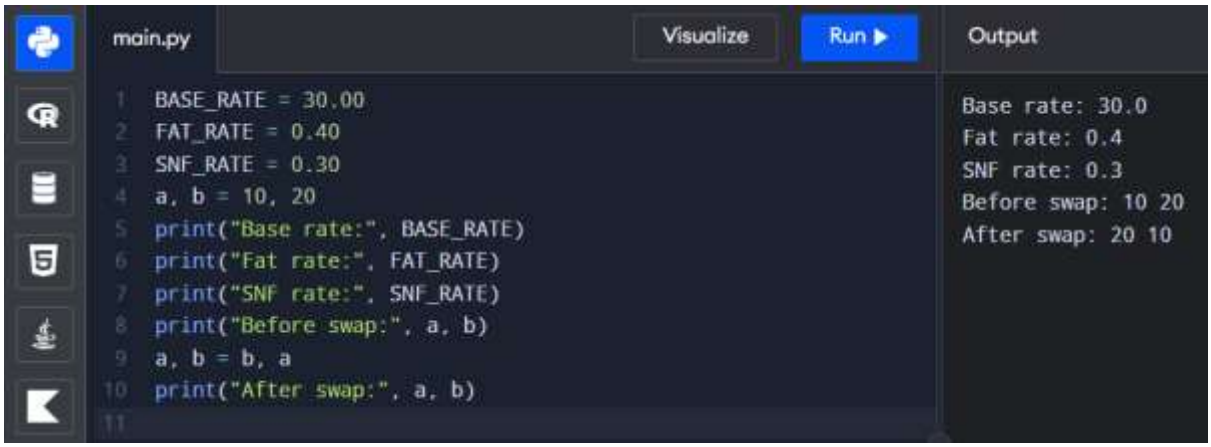
payment = quantity * rate

DISPLAY payment

END

Test case: Quantity = 5 litres, Rate = ₹30; Expected payment = ₹150.

Result: Variables and constants are used appropriately.



Task 4 – Arithmetic Expressions for Payment

Aim: To calculate the total payment using arithmetic expressions.

Theory: Arithmetic operators perform mathematical calculations. Multiplication calculates the basic payment, while addition can include bonuses and subtraction can account for deductions. Parentheses help establish the correct order of operations.

Algorithm:

1. Obtain the milk quantity and rate.
2. Calculate the basic payment.
3. Add any applicable bonus.
4. Subtract any applicable deduction.
5. Display the final payment.

Pseudocode:

BEGIN

INPUT quantity

INPUT rate

INPUT bonus

INPUT deduction

basic_payment = quantity * rate

total_payment = basic_payment + bonus - deduction

DISPLAY total_payment

END

Test case: Quantity = 10 litres, Rate = ₹40, Bonus = ₹20, Deduction = ₹10; Expected payment = ₹410.

Result: Arithmetic expressions calculate the final payment.



Task 5 – Input and Validation

Aim: To accept user input and validate the entered values.

Theory: Input allows the user to provide milk quantity and rate. Validation ensures that the values are meaningful before calculation. Negative quantities and negative rates should be rejected. Invalid entries should produce a suitable error message.

Algorithm:

1. Read the quantity and rate.
2. Check whether the values are valid.
3. If invalid, display an error message.
4. Otherwise, calculate the payment.
5. Display the result.

Pseudocode:

BEGIN

INPUT quantity

INPUT rate

IF quantity < 0 OR rate < 0 THEN

DISPLAY "Invalid input"

ELSE

payment = quantity * rate

DISPLAY payment

END IF

END

Test case: Quantity = −5 litres, Rate = ₹40; Expected output = Invalid input.

Result: Input validation prevents invalid values from being used in the calculation.



Task 6 – User-Defined Functions

Aim: To organise the payment calculation using a user-defined function.

Theory: A function is a reusable block of code that performs a specific task. Functions improve modularity, reduce repetition and make programs easier to test. Parameters provide input values, and a return statement sends the calculated result back to the calling part of the program.

Algorithm:

1. Define a payment calculation function.
2. Accept quantity and rate as parameters.
3. Calculate the payment.
4. Return the result.
5. Display the returned value.

Pseudocode:

BEGIN

DEFINE calculate_payment(quantity, rate)

payment = quantity * rate

RETURN payment

END FUNCTION

INPUT quantity

INPUT rate

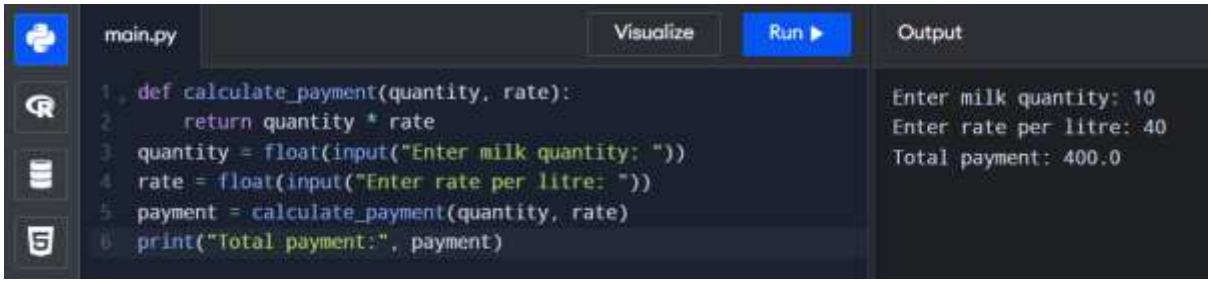
result = calculate_payment(quantity, rate)

DISPLAY result

END

Test case: Quantity = 8 litres, Rate = ₹35; Expected payment = ₹280.

Result: The calculation is organised into a reusable function.



Task 7 – Modules and Import Statements

Aim: To understand how modules support code reuse and organisation.

Theory: A module is a file containing reusable functions, classes or variables. The import statement allows a program to access functionality provided by another module. For example, Python's built-in decimal module supports decimal arithmetic useful for financial calculations.

Algorithm:

1. Identify the required functionality.
2. Import the suitable module.
3. Use the module's functionality.
4. Calculate and display the payment.

Pseudocode:

BEGIN

IMPORT required module

INPUT quantity

INPUT rate

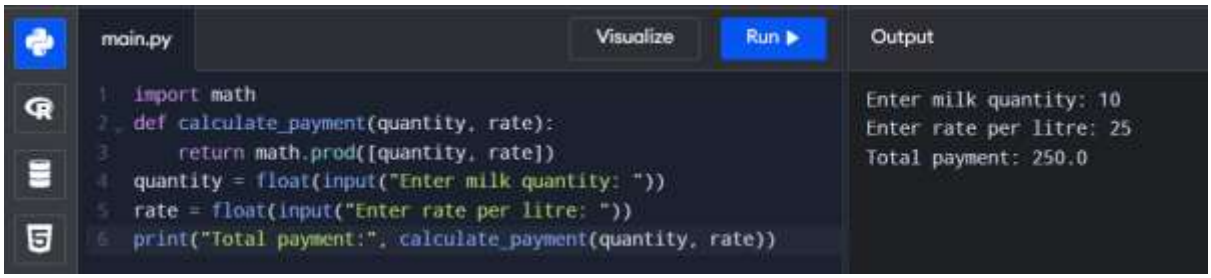
CALCULATE payment using suitable numeric operations

DISPLAY payment

END

Test case: Quantity = 10 litres, Rate = ₹25; Expected payment = ₹250.

Result: Modules help organise programs and reuse existing functionality.



Task 8 – Algorithm and Hand Tracing

Aim: To develop an algorithm and manually trace the payment calculation.

Theory: An algorithm is a finite sequence of steps used to solve a problem. Hand tracing means manually following each step using sample input values. It helps identify logical errors before executing the program.

Algorithm:

1. Start with the input values.
2. Multiply quantity by rate.
3. Store the calculated payment.
4. Display the payment.
5. Verify the result manually.

Pseudocode:

BEGIN

quantity = 12

rate = 30

payment = quantity * rate

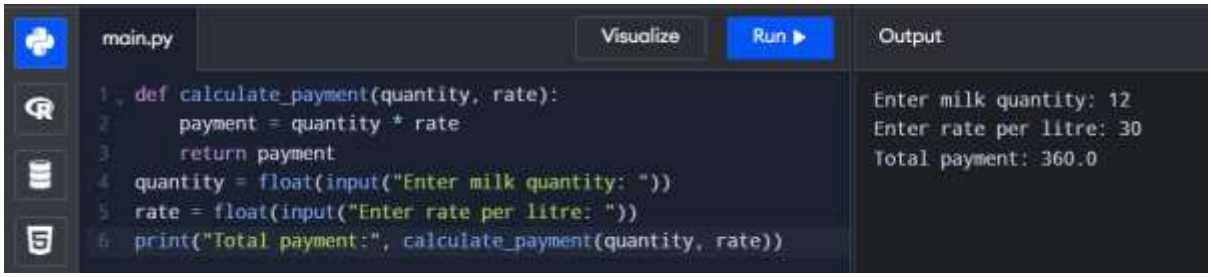
DISPLAY payment

END

Step	Operation	Value
1	Quantity	12
2	Rate	30
3	12 × 30	360

Expected output: 360

Result: Hand tracing verifies the calculation logic.



Task 9 – Testing and Test Cases

Aim: To test the payment calculator using different input values.

Theory: Testing checks whether a program produces the expected output. Normal test cases verify valid inputs, boundary cases check values such as zero, and invalid test cases verify error handling. Testing improves the reliability of the payment system.

Algorithm:

1. Prepare different test cases.
2. Enter the test inputs.
3. Execute the program.
4. Compare actual output with expected output.
5. Identify and correct errors.

Pseudocode:

BEGIN

FOR EACH test_case

PROVIDE input values

CALCULATE payment

COMPARE actual output WITH expected output

DISPLAY test result

END FOR

END

Test cases:

Quantity	Rate	Expected result
10	40	₹400
0	40	₹0
5	30	₹150
−5	40	Invalid input

Result: Different test cases are used to verify the correctness of the program.



Task 10 – Performance Comparison: Float and Decimal

Aim: To compare floating-point and decimal arithmetic for payment calculations.

Theory: The float type stores numbers in binary floating-point representation, so some decimal values cannot be represented exactly. The Decimal type supports decimal arithmetic and is often preferable for financial calculations. Performance comparison measures execution time, but correctness and precision are also important.

Algorithm:

1. Prepare the same set of numerical values.
2. Perform calculations using floating-point arithmetic.
3. Perform calculations using decimal arithmetic.
4. Compare the results and execution times.
5. Choose the appropriate representation.

Pseudocode:

BEGIN

PREPARE test values

CALCULATE result using FLOAT

CALCULATE result using DECIMAL

COMPARE numerical results

MEASURE execution times

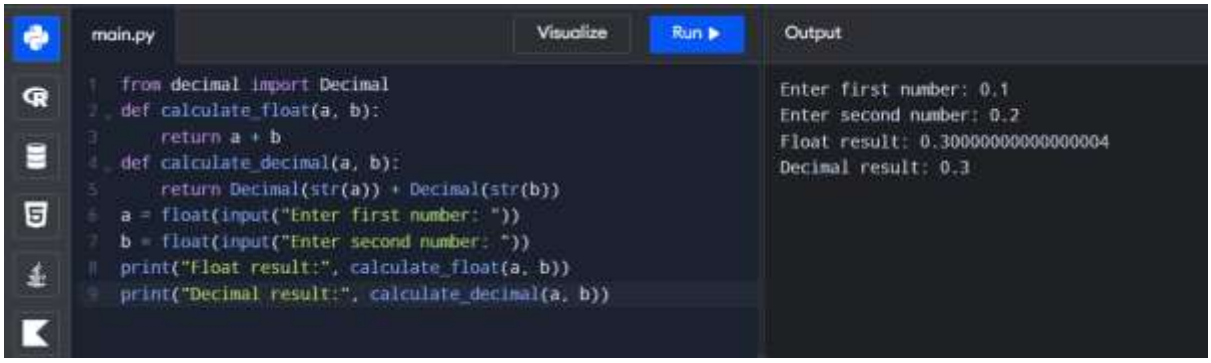
DISPLAY comparison

END

Test case: Calculate 0.1 + 0.2.

Expected observation: Floating-point arithmetic may produce a small representation difference, while decimal arithmetic can represent these decimal inputs exactly when constructed appropriately.

Result: The numerical representations are compared for accuracy and performance.



Task 11 – System Design Recommendation

Aim: To design a reliable and maintainable milk payment calculator.

Theory: A good system design separates input handling, validation, calculation and output. Modular design makes the program easier to test and maintain. Financial calculations should use appropriate precision, and invalid inputs should be handled safely.

Algorithm:

1. Accept the farmer's details and milk quantity.
2. Validate the input.
3. Obtain the applicable rate.
4. Calculate the payment.

5. Display the payment details.
6. Handle invalid entries appropriately.

Pseudocode:

BEGIN

INPUT farmer_details

INPUT quantity

INPUT rate

IF input is invalid THEN

DISPLAY error message

ELSE

payment = quantity * rate

DISPLAY farmer_details

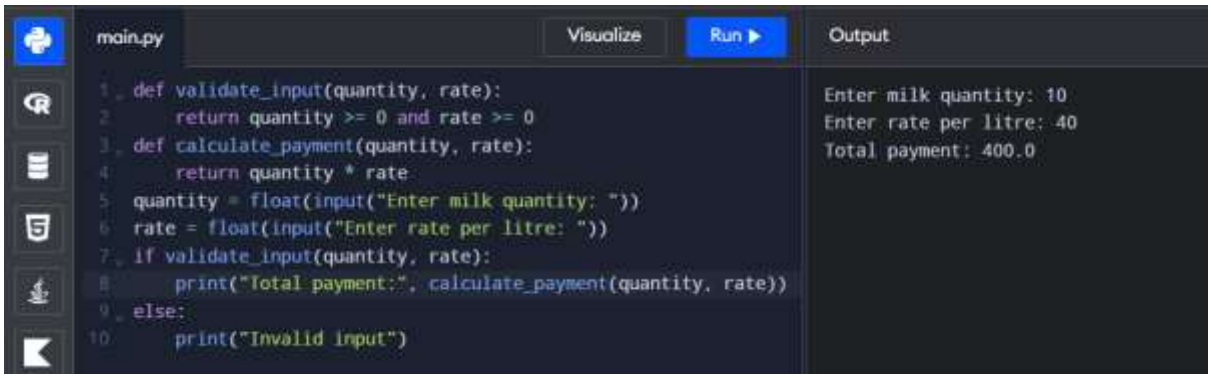
DISPLAY payment

END IF

END

Test case: Valid quantity = 10 litres, Rate = ₹40; Expected payment = ₹400.

Result: A modular and reliable system design is proposed.



Task 12 – Seasonal Rate and Bonus Calculation

Aim: To calculate milk payment based on seasonal rates and applicable bonuses.

Theory: A milk cooperative may use different rates during different seasons. A bonus can be added when a farmer meets the specified conditions. The program must select the correct rate, calculate the basic payment and add the eligible bonus. The conditions and bonus values should follow the cooperative's rules.

Algorithm:

1. Read the total milk quantity.
2. Read the season.
3. Select the applicable rate.
4. Calculate the basic payment.
5. Check bonus eligibility.
6. Add the eligible bonus.
7. Display the final payment.

Pseudocode:

```
BEGIN
   INPUT quantity
   INPUT season
 IF season = "Summer" THEN
      rate = summer_rate
   ELSE
      rate = regular_rate
   END IF
   basic_payment = quantity * rate
   IF bonus_condition is satisfied THEN
      total_payment = basic_payment + bonus
   ELSE
      total_payment = basic_payment
   END IF
```

DISPLAY total_payment

END

Test case: Quantity = 10 litres, Summer rate = ₹40, Eligible bonus = ₹20; Expected payment = ₹420.

Result: Seasonal rates and bonus conditions are included in the payment calculation.

